

Quick Start

This tutorial will teach you how to install `hivemind`, host your own experts and train them remotely.

Installation

Just `pip install hivemind` to get the latest release (requires Python 3.8 or newer).

You can also install the bleeding edge version from GitHub:

```
git clone https://github.com/learning-at-home/hivemind
cd hivemind
pip install -e . --no-use-pep517
```

Decentralized Training

Hivemind is a set of building blocks for decentralized training. In this tutorial, we'll use two of these blocks to train a simple neural network to classify CIFAR-10 images. We assume that you are already familiar with the official [CIFAR-10 example](#) from the PyTorch website.

We build on top of the official example to spin up distributed training of a two-layer neural network by averaging weights. For simplicity, this tutorial will use two non-GPU peers running on the same machine. If you try to run this example on two separate machines with different IPs, this example will not work. To read more about how to perform training on more than one machine check out [DHT - Running Across the Internet](#). If you get to the end of this tutorial, we'll give you an example of actual distributed training of Transformers ;)

For now, let's run our first training peer:

```

import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import datasets, transforms
from tqdm.auto import tqdm

import hivemind

# Create dataset and model, same as in the basic tutorial
# For this basic tutorial, we download only the training set
transform = transforms.Compose(
    [transforms.ToTensor(), transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

trainset = datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)

model = nn.Sequential(nn.Conv2d(3, 16, (5, 5)), nn.MaxPool2d(2, 2), nn.ReLU(),
                      nn.Conv2d(16, 32, (5, 5)), nn.MaxPool2d(2, 2), nn.ReLU(),
                      nn.Flatten(), nn.Linear(32 * 5 * 5, 10))
opt = torch.optim.SGD(model.parameters(), lr=0.001, momentum=0.9)

# Create DHT: a decentralized key-value storage shared between peers
dht = hivemind.DHT(start=True)
print("To join the training, use initial_peers =", [str(addr) for addr in
dht.get_visible_maddrs()])

# Set up a decentralized optimizer that will average with peers in background
opt = hivemind.Optimizer(
    dht=dht,                # use a DHT that is connected with other peers
    run_id='my_cifar_run',  # unique identifier of this collaborative run
    batch_size_per_step=32, # each call to opt.step adds this many samples towards the next
epoch
    target_batch_size=10000, # after peers collectively process this many samples, average
weights and begin the next epoch
    optimizer=opt,          # wrap the SGD optimizer defined above
    use_local_updates=True, # perform optimizer steps with local gradients, average parameters
in background
    matchmaking_time=3.0,   # when averaging parameters, gather peers in background for up to
this many seconds
    averaging_timeout=10.0, # give up on averaging if not successful in this many seconds
    verbose=True           # print logs incessantly
)

# Note: if you intend to use GPU, switch to it only after the decentralized optimizer is created
with tqdm() as progressbar:
    while True:
        for x_batch, y_batch in torch.utils.data.DataLoader(trainset, shuffle=True,
batch_size=32):
            opt.zero_grad()
            loss = F.cross_entropy(model(x_batch), y_batch)
            loss.backward()
            opt.step()

            progressbar.desc = f"loss = {loss.item():.3f}"
            progressbar.update()

```

As you can see, this code is regular PyTorch with one notable exception: it wraps your regular optimizer with a `hivemind.Optimizer`. This optimizer uses `DHT` to find other peers and tries to exchange parameters with them. When you run the code (please do so), you will see the following output:

```
To join the training, use initial_peers = ['/ip4/127.0.0.1/tcp/XXX/p2p/YYY']
[...] Starting a new averaging round with current parameters.
```

This is `hivemind.Optimizer` telling you that it's looking for peers. Since there are no peers, we'll need to create them ourselves.

Copy the entire script (or notebook) and modify this line:

```
# old version:
dht = hivemind.DHT(start=True)

# new version: added initial_peers
dht = hivemind.DHT(initial_peers=['/ip4/127.0.0.1/tcp/COPY_FULL_ADDRESS_FROM_PEER1_OUTPUTS'],
start=True)
```

► Here's the full code of the second peer

Instead of setting up a new DHT, the second peer will link up with the existing DHT node from the first peer. If you run the second peer, you will see that both first and second peer will periodically report averaging parameters:

```
[...] Starting a new averaging round with current parameters.
[...] Finished averaging round in with 2 peers.
```

This message means that the optimizer has averaged model parameters with another peer in background and applied them during one of the calls to `opt.step()`. You can start more peers by replicating the same code as the second peer, using either the first or second peer as `initial_peers`.

Each new peer starts with an untrained network and must download the latest training state before it can contribute. By default, peer will automatically detect that it is out of sync and start `Downloading parameters from peer <...>`. To avoid wasting the first optimizer step, one can manually download the latest model/optimizer state right before it begins training  [latest](#) ▼ minibatches:

```
opt.load_state_from_peers()
```

Congrats, you've just started a pocket-sized experiment with decentralized deep learning!

However, this is only the basics of what hivemind can do. In [this example](#), we show how to use a more advanced version of DecentralizedOptimizer to collaboratively train a large Transformer over the internet.

If you want to learn more about each individual component,

- Learn how to use `hivemind.DHT` using this basic [DHT tutorial](#),
- Read more on how to use `hivemind.Optimizer` in its [documentation page](#),
- Learn the underlying math behind `hivemind.Optimizer` in [Diskin et al., \(2021\)](#), [Li et al. \(2020\)](#) and [Ryabinin et al. \(2021\)](#).
- Read about setting up Mixture-of-Experts training in [this guide](#),

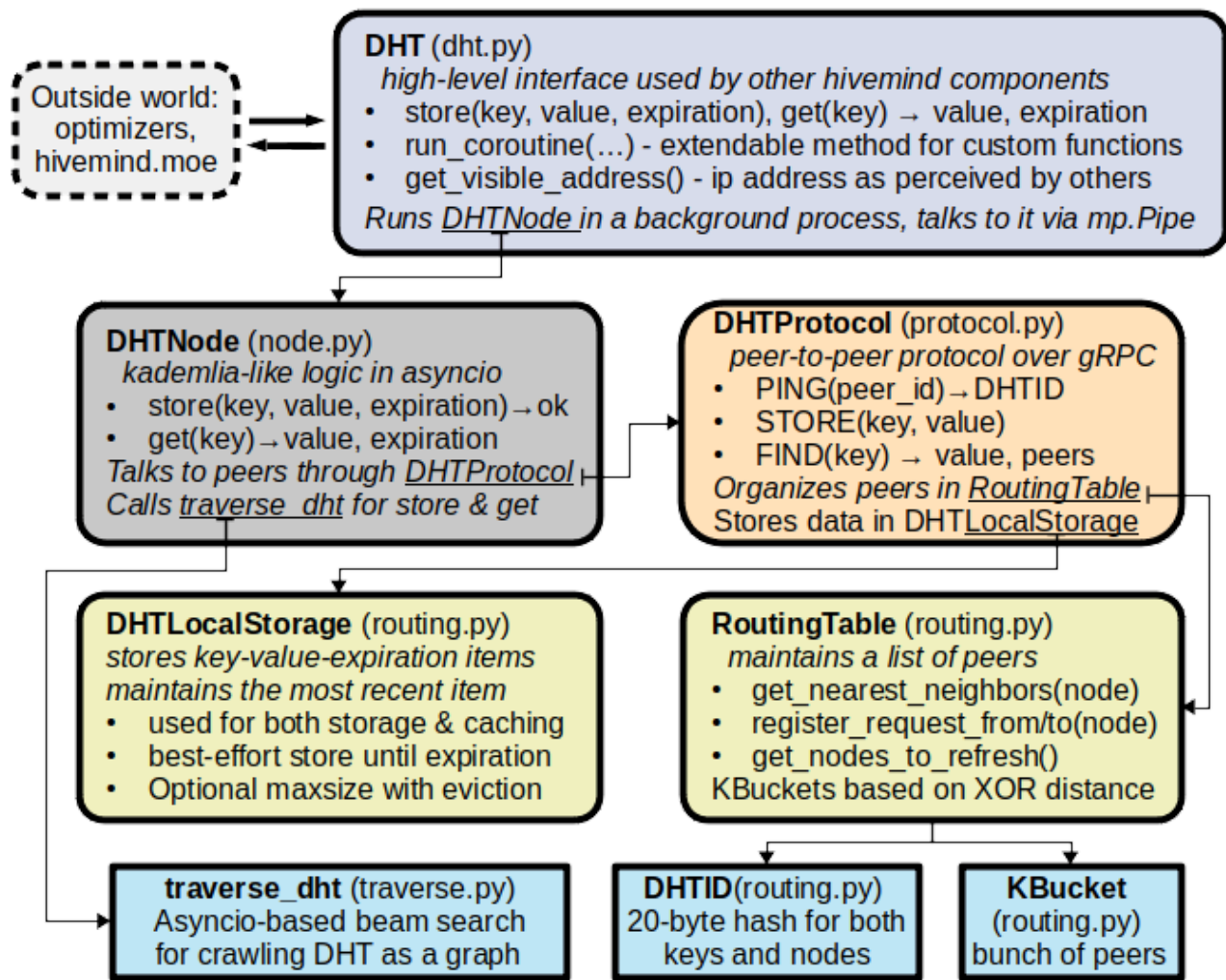
hivemind.dht

This is a Distributed Hash Table optimized for rapidly accessing a lot of lightweight metadata. Hivemind DHT is based on Kademlia [1] with added support for improved bulk store/get operations and caching.

The code is organized as follows:

- **class DHT (dht.py)** - high-level class for model training. Runs DHTNode in a background process.
 - **class DHTNode (node.py)** - an asyncio implementation of dht server, stores AND gets keys.
 - **class DHTProtocol (protocol.py)** - an RPC protocol to request data from dht nodes.
 - **async def traverse_dht (traverse.py)** - a search algorithm that crawls DHT peers.
-
- [1] Maymounkov P., Mazieres D. (2002) Kademlia: A Peer-to-Peer Information System Based on the XOR Metric.
 - [2] <https://github.com/bmuller/kademlia> , Brian, if you're reading this: THANK YOU! you're awesome :)

Here's a high level scheme of how these components interact with one another:



DHT and DHTNode

```

class hivemind.dht.DHT(initial_peers: Optional[Sequence[Union[Multiaddr, str]]] = None, *, start: bool, p2p:
Optional[P2P] = None, daemon: bool = True, num_workers: int = 4, record_validators:
Iterable[RecordValidatorBase] = (), shutdown_timeout: float = 3, await_ready: bool = True, **kwargs)
    [source]
  
```

A high-level interface to a hivemind DHT that runs a single DHT node in a background process. * hivemind servers periodically announce their experts via `declare_experts` (`dht_handler.py`) * trainers find most suitable experts via `RemoteMixtureOfExperts` (`beam_search.py`)

Parameters

- **initial_peers** – multiaddrs of one or more active DHT peers (if you want to join an existing DHT)
 - **start** – if True, automatically starts the background process on creation. Otherwise await manual start
 - **daemon** – if True, the background process is marked as daemon and ;  [latest](#) ▼
- terminated after main process

- **num_workers** – declare_experts and get_experts will use up to this many parallel workers (but no more than one per key)
- **expiration** – experts declared from this node expire after this many seconds (default = 5 minutes)
- **record_validators** – instances of RecordValidatorBase used for signing and validating stored records. The validators will be combined using the CompositeValidator class. It merges them when possible (according to their .merge_with() policies) and orders them according to the .priority properties.
- **shutdown_timeout** – when calling .shutdown, wait for up to this many seconds before terminating
- **await_ready** – if True, the constructor waits until the DHT process is ready to process incoming requests
- **kwargs** – any other params will be forwarded to DHTNode and hivemind.p2p.P2P upon creation

`run()→ None` [\[source\]](#)

Serve DHT forever. This function will not return until DHT node is shut down

`run_in_background(await_ready: bool = True, timeout: Optional[float] = None)→ None` [\[source\]](#)

Starts DHT in a background process. if await_ready, this method will wait until background dht is ready to process incoming requests or for :timeout: seconds max.

`shutdown()→ None` [\[source\]](#)

Shut down a running dht process

`get(key: Any, latest: bool = False, return_future: bool = False, **kwargs)→ Union[ValueWithExpiration[Any], None, MPFuture]` [\[source\]](#)

Search for a key across DHT and return either first or latest entry (if found). :param key: same key as in node.store(...) :param latest: if True, finds the latest value, otherwise finds any non-expired value (which is much faster) :param return_future: if False (default), return when finished. Otherwise return MPFuture and run in background. :param kwargs: parameters forwarded to DHTNode.get_many_by_id :returns: (value, expiration time); if value was not found, returns None

`store(key: Any, value: Any, expiration_time: float, subkey: Optional[Any] = None, return_future: bool = False, **kwargs)→ Union[bool, MPFuture]` [\[source\]](#)

 [latest](#) ▼

Find num_replicas best nodes to store (key, value) and store it there until expiration time.

Parameters

- **key** – msgpack-serializable key to be associated with value until expiration.
- **value** – msgpack-serializable value to be stored under a given key until expiration.
- **expiration_time** – absolute time when the entry should expire, based on `hivemind.get_dht_time()`
- **subkey** – if specified, add a value under that subkey instead of overwriting key (see `DHTNode.store_many`)
- **return_future** – if False (default), return when finished. Otherwise return `MPFuture` and run in background.

Returns

True if store succeeds, False if it fails (due to no response or newer value)

```
run_coroutine(coro: Callable[[DHT, DHTNode], Awaitable[ReturnType]], return_future: bool = False) → Union[ReturnType, MPFuture[ReturnType]] \[source\]
```

Execute an asynchronous function on a DHT participant and return results. This is meant as an interface

for running custom functions DHT for special cases (e.g. declare experts, beam search)

Parameters

- **coro** – async function to be executed. Receives 2 arguments: this DHT daemon and a running `DHTNode`
- **return_future** – if False (default), return when finished. Otherwise return `MPFuture` and run in background.

Returns

coroutine outputs or `MPFuture` for these outputs

Note

the coroutine will be executed inside the DHT process. As such, any changes to global variables or DHT fields made by this coroutine will not be accessible from the host process.

Note

all time-consuming operations in `coro` should be asynchronous (e.g. `asyncio.sleep` instead of `time.sleep`) or use `asyncio.get_event_loop().run_in_executor`  [latest](#) ▼
coroutine from blocking background DHT tasks

Note

when `run_coroutine` is called with `return_future=False`, `MPFuture` can be cancelled to interrupt the task.

`get_visible_maddrs(latest: bool = False) → List[Multiaddr]` [\[source\]](#)

Get multiaddrs of the current DHT node that should be accessible by other peers.

Parameters

latest – ask the P2P daemon to refresh the visible multiaddrs

`async replicate_p2p() → P2P` [\[source\]](#)

Get a replica of a P2P instance used in the DHT process internally. The replica uses the same P2P daemon as the DHT and only works while DHT is alive.

`class hivemind.dht.DHTNode(*, _initialized_with_create=False)` [\[source\]](#)

Asyncio-based class that represents one DHT participant. Created via `await DHTNode.create(...)` Each `DHTNode` has an identifier, a local storage and access too other nodes via `DHTProtocol`.

Note

Hivemind DHT is optimized to store a lot of temporary metadata that is regularly updated. For example, expert heartbeat emitted by a `hivemind.moe.Server` responsible for that expert. Such metadata does not require regular maintenance by peers or persistence on shutdown. Instead, `DHTNode` is designed to rapidly send bulk data and resolve conflicts.

Every (key, value) pair in this DHT has an expiration time - float computed as `get_dht_time()` (UnixTime by default) DHT nodes always prefer values with higher expiration time and may delete any value past its expiration.

Similar to Kademlia RPC protocol, hivemind DHT has 3 RPCs:

- ping - request peer's identifier and update routing table (same as Kademlia PING RPC)
- store - send several (key, value, expiration_time) pairs to the same peer (like Kademlia STORE, but in bulk)
- find - request one or several keys, get values and expiration (if peer finds it locally) and :bucket_size: of

nearest peers from recipient's routing table (ordered nearest-to-farth recipient itself) This RPC is a mixture between Kademlia `FIND_NODE` and `FIND_VALUE` with multiple keys per call.

 [latest](#) ▼

A DHTNode follows the following contract:

- when asked to get(key), a node must find and return a value with highest expiration time that it found across DHT IF that time has not come yet. if expiration time is smaller than current get_dht_time(), node may return None;
- when requested to store(key: value, expiration_time), a node must store (key => value) at until expiration time or until DHTNode gets the same key with greater expiration time. If a node is asked to store a key but it already has the same key with newer expiration, store will be rejected. Store returns True if accepted, False if rejected;
- when requested to store(key: value, expiration_time, subkey=subkey), adds a sub-key to a dictionary value type. Dictionary values can have multiple sub-keys stored by different peers with individual expiration times. A subkey will be accepted to a dictionary either if there is no such sub-key or if new subkey's expiration is later than previous expiration under that subkey. See DHTProtocol.call_store for details.

DHTNode also features several (optional) caching policies:

- cache_locally: after GET, store the result in node's own local cache
- cache_nearest: after GET, send the result to this many nearest nodes that don't have that value yet (see Kademlia)
- cache_on_store: after STORE, either save or remove that key from node's own cache depending on store status
- cache_refresh_before_expiry: if a value in cache was used and is about to expire, try to GET it this many seconds before expiration. The motivation here is that some frequent keys should be always kept in cache to avoid latency.
- reuse_get_requests: if there are several concurrent GET requests, when one request finishes, DHTNode will attempt to reuse the result of this GET request for other requests with the same key. Useful for batch-parallel requests.

```
async classmethod create(p2p: Optional[P2P] = None, node_id: Optional[DHTID] = None, initial_peers: Optional[Sequence[Union[Multiaddr, str]]] = None, bucket_size: int = 20, num_replicas: int = 5, depth_modulo: int = 5, parallel_rpc: int = None, wait_timeout: float = 3, refresh_timeout: Optional[float] = None, bootstrap_timeout: Optional[float] = None, cache_locally: bool = True, cache_nearest: int = 1, cache_size=None, cache_refresh_before_expiry: float = 5, cache_on_store: bool = True, reuse_get_requests: bool = True, num_workers: int = 4, chunk_size: int = 16, blacklist_time: float = 5.0, backoff_rate: float = 2.0, client_mode: bool = False, record_validator: Optional[RecordValidatorBase] = None, authorizer: Optional[AuthorizerBase] = None, ensure_bootstrap_success: bool = True, strict: bool = True, **kwargs)→ DHTNode [source]
```

Parameters

- **p2p** – instance of hivemind.p2p.P2P that will be used for commun  latest ▼
DHTNode will create and manage its own P2P instance with given initial_peers and parameters from kwargs

- **node_id** – current node's DHTID for hivemind.dht, determines which keys it will store locally, defaults to random id
- **initial_peers** – multiaddrs of one or more active DHT peers (if you want to join an existing DHT)
- **bucket_size** – max number of nodes in one k-bucket (k). Trying to add {k+1}st node will cause a bucket to either split in two buckets along the midpoint or reject the new node (but still save it as a replacement) Recommended value: k is chosen s.t. any given k nodes are very unlikely to all fail after staleness_timeout
- **num_replicas** – number of nearest nodes that will be asked to store a given key, default = bucket_size ($\approx k$)
- **depth_modulo** – split full k-bucket if it contains root OR up to the nearest multiple of this value ($\approx b$)
- **parallel_rpc** – maximum number of concurrent outgoing RPC requests emitted by DHTProtocol Reduce this value if your RPC requests register no response despite the peer sending the response.
- **wait_timeout** – a kademia rpc request is deemed lost if we did not receive a reply in this many seconds
- **refresh_timeout** – refresh buckets if no node from that bucket was updated in this many seconds if staleness_timeout is None, DHTNode will not refresh stale buckets (which is usually okay)
- **bootstrap_timeout** – after one of peers responds, await other peers for at most this many seconds
- **cache_locally** – if True, caches all values (stored or found) in a node-local cache
- **cache_on_store** – if True, update cache entries for a key after storing a new item for that key
- **cache_nearest** – whenever DHTNode finds a value, it will also store (cache) this value on this many nearest nodes visited by search algorithm. Prefers nodes that are nearest to :key: but have no value yet
- **cache_size** – if specified, local cache will store up to this many records (as in LRU cache)
- **cache_refresh_before_expiry** – if nonzero, refreshes locally cached values if they are accessed this many seconds before expiration time.
- **reuse_get_requests** – if True, DHTNode allows only one traverse_  latest ▼
for every key all concurrent get requests for the same key will reuse the procedure that is currently in progress

- **num_workers** – concurrent workers in `traverse_dht` (see `traverse_dht` `num_workers` param)
- **chunk_size** – maximum number of concurrent calls in `get_many` and cache refresh queue
- **blacklist_time** – excludes non-responsive peers from search for this many seconds (set 0 to disable)
- **backoff_rate** – blacklist time will be multiplied by `:backoff_rate`: for each successive non-response
- **ensure_bootstrap_success** – raise an error if node could not connect to initial peers (or vice versa) If False, print a warning instead. It is recommended to keep this flag unless you know what you're doing.
- **strict** – if True, any error encountered in validation will interrupt the creation of `DHTNode`
- **client_mode** – if False (default), this node will accept incoming requests as a full DHT “citizen” if True, this node will refuse any incoming requests, effectively being only a client
- **record_validator** – instance of `RecordValidatorBase` used for signing and validating stored records
- **authorizer** – instance of `AuthorizerBase` used for signing and validating requests and response for a given authorization protocol
- **kwargs** – extra parameters for an internally created instance of `hivemind.p2p.P2P`. Should be empty if the P2P instance is provided in the constructor

`async shutdown()` [\[source\]](#)

Process existing requests, close all connections and stop the server

`async find_nearest_nodes(queries: Collection[DHTID], k_nearest: Optional[int] = None, beam_size: Optional[int] = None, num_workers: Optional[int] = None, node_to_peer_id: Optional[Dict[DHTID, PeerID]] = None, exclude_self: bool = False, **kwargs) → Dict[DHTID, Dict[DHTID, PeerID]]` [\[source\]](#)

Parameters

- **queries** – find k nearest nodes for each of these DHTIDs
- **k_nearest** – return this many nearest nodes for every query (if there are enough nodes)
- **beam_size** – replacement for `self.beam_size`, see `traverse_dht` `beam_size` param

- **num_workers** – replacement for self.num_workers, see traverse_dht num_workers param
- **node_to_peer_id** – if specified, uses this dict[node_id => peer_id] as initial peers
- **exclude_self** – if True, nearest nodes will not contain self.node_id (default = use local peers)
- **kwargs** – additional params passed to traverse_dht

Returns

for every query, return nearest peers ordered dict[peer DHTID -> network PeerID], nearest-first

```
async store(key: Any, value: Any, expiration_time: float, subkey: Optional[Any] = None, **kwargs)→
bool \[source\]
```

Find num_replicas best nodes to store (key, value) and store it there at least until expiration time. :note: store is a simplified interface to store_many, all kwargs are forwarded there :returns: True if store succeeds, False if it fails (due to no response or newer value)

```
async store_many(keys: List[Any], values: List[Any], expiration_time: Union[float, List[float]], subkeys:
Optional[Union[Any, List[Optional[Any]]]] = None, exclude_self: bool = False, await_all_replicas=True,
**kwargs)→ Dict[Any, bool] \[source\]
```

Traverse DHT to find up :num_replicas: to best nodes to store multiple (key, value, expiration_time) pairs.

Parameters

- **keys** – arbitrary serializable keys associated with each value
- **values** – serializable “payload” for each key
- **expiration_time** – either one expiration time for all keys or individual expiration times (see class doc)
- **subkeys** – an optional list of same shape as keys. If specified, this
- **kwargs** – any additional parameters passed to traverse_dht function (e.g. num workers)
- **exclude_self** – if True, never store value locally even if you are one of the nearest nodes
- **await_all_replicas** – if False, this function returns after first store_  [latest](#) 
in background if True, the function will wait for num_replicas successful stores or running out of beam_size nodes

Note

if `exclude_self` is `True` and `self.cache_locally == True`, value will still be `__cached__` locally

Returns

for each key: `True` if store succeeds, `False` if it fails (due to no response or newer value)

```
async get(key: Any, latest=False, **kwargs)→ Optional[ValueWithExpiration[Any]] \[source\]
```

Search for a key across DHT and return either first or latest entry (if found). :param key: same key as in `node.store(...)` :param latest: if `True`, finds the latest value, otherwise finds any non-expired value (which is much faster) :param kwargs: parameters forwarded to `get_many_by_id` :returns: (value, expiration time); if value was not found, returns `None`

```
async get_many(keys: Collection[Any], sufficient_expiration_time: Optional[float] = None, **kwargs)→ Dict[Any, Union[ValueWithExpiration[Any], None, Awaitable[Optional[ValueWithExpiration[Any]]]]] \[source\]
```

Traverse DHT to find a list of keys. For each key, return latest (value, expiration) or `None` if not found.

Parameters

- **keys** – traverse the DHT and find the value for each of these keys (or `(None, None)` if not key found)
- **sufficient_expiration_time** – if the search finds a value that expires after this time, default = time of call, find any value that did not expire by the time of call If `min_expiration_time=float('inf')`, this method will find a value with `_latest_expiration`
- **kwargs** – for full list of parameters, see `DHTNode.get_many_by_id`

Returns

for each key: value and its expiration time. If nothing is found, returns `(None, None)` for that key

Note

in order to check if get returned a value, please check if `(expiration_time is None)`

```
async get_many_by_id(key_ids: Collection[DHTID], sufficient_expiration_time: Optional[float] = None, num_workers: Optional[int] = None, beam_size: Optional[int] = None, return_futures: bool = False, _is_refresh=False)→ Dict[DHTID, Union[ValueWithExpiration[Any], None, Awaitable[Optional[ValueWithExpiration[Any]]]]] \[source\]  latest ▼
```

Traverse DHT to find a list of DHTIDs. For each key, return latest (value, expiration) or None if not found.

Parameters

- **key_ids** – traverse the DHT and find the value for each of these keys (or (None, None) if not key found)
- **sufficient_expiration_time** – if the search finds a value that expires after this time, default = time of call, find any value that did not expire by the time of call If `min_expiration_time=float('inf')`, this method will find a value with `_latest_expiration`
- **beam_size** – maintains up to this many nearest nodes when crawling dht, default `beam_size = bucket_size`
- **num_workers** – override for default `num_workers`, see `traverse_dht num_workers` param
- **return_futures** – if True, immediately return `asyncio.Future` for every before interacting with the network. The algorithm will populate these futures with (value, expiration) when it finds the corresponding key Note: canceling a future will stop search for the corresponding key
- **_is_refresh** – internal flag, set to True by an internal cache refresher (if enabled)

Returns

for each key: value and its expiration time. If nothing is found, returns (None, None) for that key

Note

in order to check if get returned a value, please check (`expiration_time` is None)

DHT communication protocol

RPC protocol that provides nodes a way to communicate with each other

```
class hivemind.dht.protocol.DHTProtocol(*, _initialized_with_create=False) \[source\]
```

```
    serializer \[source\]
```

```
    alias of MSGPackSerializer
```

```
    async classmethod create(p2p: P2P, node_id: DHTID, bucket_size: int, depth_modulo: int, num_replicas:
int, wait_timeout: float, parallel_rpc: Optional[int] = None, cache_size: Optional[int] =
client_mode: bool = False, record_validator: Optional[RecordValidatorBase] = None, ai
Optional[AuthorizerBase] = None)→ DHTProtocol \[source\]
```

 [latest](#) ▼

A protocol that allows DHT nodes to request keys/neighbors from other DHT nodes. As a side-effect, DHTProtocol also maintains a routing table as described in <https://pdos.csail.mit.edu/~petar/papers/maymounkov-kademlia-lncs.pdf>

See DHTNode (node.py) for a more detailed description.

Note

the `rpc_*` methods defined in this class will be automatically exposed to other DHT nodes, for instance, `def rpc_ping` can be called as `protocol.call_ping(peer_id, dht_id)` from a remote machine Only the `call_*` methods are meant to be called publicly, e.g. from DHTNode Read more: <https://github.com/bmuller/rpcudp/tree/master/rpcudp>

`get_stub(peer: PeerID)→ AuthRPCWrapper` [\[source\]](#)

get a stub that sends requests to a given peer

`async call_ping(peer: PeerID, validate: bool = False, strict: bool = True)→ Optional[DHTID]` [\[source\]](#)

Get peer's node id and add him to the routing table. If peer doesn't respond, return None
:param peer: peer ID to ping
:param validate: if True, validates that node's peer_id is available
:param strict: if strict=True, validation will raise exception on fail, otherwise it will only warn
:note: if DHTProtocol was created with `client_mode=False`, also request peer to add you to his routing table

Returns

node's DHTID, if peer responded and decided to send his node_id

`async rpc_ping(request: PingRequest, context: P2PContext)→ PingResponse` [\[source\]](#)

Some node wants us to add it to our routing table.

`async call_store(peer: PeerID, keys: Sequence[DHTID], values: Sequence[Union[bytes, DictionaryDHTValue]], expiration_time: Union[float, Sequence[float]], subkeys: Optional[Union[Any, Sequence[Optional[Any]]]] = None, in_cache: Optional[Union[bool, Sequence[bool]]] = None)→ Optional[List[bool]]` [\[source\]](#)

Ask a recipient to store several (key, value : expiration_time) items or update their older value

Parameters

- **peer** – request this peer to store the data
- **keys** – a list of N keys digested by `DHTID.generate(source=some_`  [latest](#) 
- **values** – a list of N serialized values (bytes) for each respective key

- **expiration_time** – a list of N expiration timestamps for each respective key-value pair(see `get_dht_time()`)
- **subkeys** – a list of N optional sub-keys. If None, stores value normally. If not subkey is not None: 1) if local storage doesn't have :key:, create a new dictionary {subkey: (value, expiration_time)} 2) if local storage already has a dictionary under :key:, try add (subkey, value, exp_time) to that dictionary 2) if local storage associates :key: with a normal value with smaller expiration, clear :key: and perform (1) 3) finally, if local storage currently associates :key: with a normal value with larger expiration, do nothing
- **in_cache** – a list of booleans, True = store i-th key in cache, value = store i-th key locally

Note

the difference between storing normally and in cache is that normal storage is guaranteed to be stored until expiration time (best-effort), whereas cached storage can be evicted early due to limited cache size

Returns

list of [True / False] True = stored, False = failed (found newer value or no response) if peer did not respond (e.g. due to timeout or congestion), returns None

`async rpc_store(request: StoreRequest, context: P2PContext)→ StoreResponse` [\[source\]](#)

Some node wants us to store this (key, value) pair

`async call_find(peer: PeerID, keys: Collection[DHTID])→ Optional[Dict[DHTID, Tuple[Optional[ValueWithExpiration[Union[bytes, DictionaryDHTValue]]], Dict[DHTID, PeerID]]]]` [\[source\]](#)

Request keys from a peer. For each key, look for its (value, expiration time) locally and

k additional peers that are most likely to have this key (ranked by XOR distance)

Returns

A dict key => Tuple[optional value, optional expiration time, nearest neighbors] value: value stored by the recipient with that key, or None if peer doesn't have this value
expiration time: expiration time of the returned value, None if no value was found
neighbors: a dictionary[node_id : peer_id] containing nearest neighbors from peer's routing table If peer didn't respond, returns None

`async rpc_find(request: FindRequest, context: P2PContext)→ FindResponse`

[\[source\]](#)

 latest ▼

Someone wants to find keys in the DHT. For all keys that we have locally, return value and expiration Also return :bucket_size: nearest neighbors from our routing table for each key (whether or not we found value)

```
async update_routing_table(node_id: Optional[DHTID], peer_id: PeerID, responded=True) [source]
```

This method is called on every incoming AND outgoing request to update the routing table

Parameters

- **peer_id** – sender peer_id for incoming requests, recipient peer_id for outgoing requests
- **node_id** – sender node id for incoming requests, recipient node id for outgoing requests
- **responded** – for outgoing requests, this indicated whether recipient responded or not. For incoming requests, this should always be True

```
class hivemind.dht.routing.RoutingTable(node_id: DHTID, bucket_size: int, depth_modulo: int) [source]
```

A data structure that contains DHT peers bucketed according to their distance to node_id. Follows Kademlia routing table as described in

<https://pdos.csail.mit.edu/~petar/papers/maymounkov-kademlia-lncs.pdf>

Parameters

- **node_id** – node id used to measure distance
- **bucket_size** – parameter k from Kademlia paper Section 2.2
- **depth_modulo** – parameter b from Kademlia paper Section 2.2.

Note

you can find a more detailed description of parameters in DHTNode, see node.py

```
get_bucket_index(node_id: DHTID)→ int [source]
```

Get the index of the bucket that the given node would fall into.

```
add_or_update_node(node_id: DHTID, peer_id: PeerID)→ Optional[Tuple[DHTID, PeerID]] [source]
```

Update routing table after an incoming request from :peer_id: or outgoing :peer_id:

Returns

If we cannot add `node_id` to the routing table, return the least-recently-updated node (Section 2.2)

Note

DHTProtocol calls this method for every incoming and outgoing request if there was a response. If this method returned a node to be ping-ed, the protocol will ping it to check and either move it to the start of the table or remove that node and replace it with

`split_bucket(index: int) → None` [\[source\]](#)

Split bucket range in two equal parts and reassign nodes to the appropriate half

`get(*, node_id: Optional[DHTID] = None, peer_id: Optional[PeerID] = None, default=None)` [\[source\]](#)

Find `peer_id` for a given DHTID or vice versa

`get_nearest_neighbors(query_id: DHTID, k: int, exclude: Optional[DHTID] = None) → List[Tuple[DHTID, PeerID]]` [\[source\]](#)

Find `k` nearest neighbors from routing table according to XOR distance, does NOT include `self.node_id`

Parameters

- **query_id** – find neighbors of this node
- **k** – find this many neighbors. If there aren't enough nodes in the table, returns all nodes
- **exclude** – if True, results will not contain `query_node_id` even if it is in table

Returns

a list of tuples (`node_id`, `peer_id`) for up to `k` neighbors sorted from nearest to farthest

`class hivemind.dht.routing.KBucket(lower: int, upper: int, size: int, depth: int = 0)` [\[source\]](#)

A bucket containing up to `:size:` of DHTIDs in `[lower, upper)` semi-interval. Maps DHT node ids to their `peer_ids`

`has_in_range(node_id: DHTID)` [\[source\]](#)

Check if `node_id` is between this bucket's lower and upper bounds

`add_or_update_node(node_id: DHTID, peer_id: PeerID) → bool` [\[source\]](#)

Add node to KBucket or update existing node, return True if successful, False if the bucket is full. If the bucket is full, keep track of node in a replacement list, per section 4.1 of the paper.

Parameters

- **node_id** – dht node identifier that should be added or moved to the front of bucket
- **peer_id** – network address associated with that node id

Note

this function has a side-effect of resetting KBucket.last_updated time

`request_ping_node()`→ `Optional[Tuple[DHTID, PeerID]]` [\[source\]](#)

Returns

least-recently updated node that isn't already being pinged right now – if such node exists

`split()`→ `Tuple[KBucket, KBucket]` [\[source\]](#)

Split bucket over midpoint, rounded down, assign nodes to according to their id

`class hivemind.dht.routing.DHTID(value: int)` [\[source\]](#)

`classmethod generate(source: Optional[Any] = None, nbits: int = 255)` [\[source\]](#)

Generates random uid based on SHA1

Parameters

source – if provided, converts this value to bytes and uses it as input for hashing function; by default, generates a random dhtid from :nbits: random bits

`xor_distance(other: Union[DHTID, Sequence[DHTID]])`→ `Union[int, List[int]]` [\[source\]](#)

Parameters

other – one or multiple DHTIDs. If given multiple DHTIDs as other, this function will compute distance from self to each of DHTIDs in other.

Returns

a number or a list of numbers whose binary representations equal bitwise xor between DHTIDs.

`to_bytes(length=20, byteorder='big', *, signed=False)`→ `bytes` [\[source\]](#)

A standard way to serialize DHTID into bytes

```
classmethod from_bytes(raw: bytes, byteorder='big', *, signed=False)→ DHTID \[source\]
```

reverse of to_bytes

Traverse (crawl) DHT

Utility functions for crawling DHT nodes, used to get and store keys in a DHT

```
async hivemind.dht.traverse.simple_traverse_dht(query_id: DHTID, initial_nodes: Collection[DHTID],  
beam_size: int, get_neighbors: Callable[[DHTID], Awaitable[Tuple[Collection[DHTID], bool]]], visited_nodes:  
Collection[DHTID] = ())→ Tuple[Tuple[DHTID], Set[DHTID]] \[source\]
```

Traverse the DHT graph using `get_neighbors` function, find `:beam_size:` nearest nodes according to `DHTID.xor_distance`.

Note

This is a simplified (but working) algorithm provided for documentation purposes. Actual `DHTNode` uses `traverse_dht` - a generalization of this algorithm that allows multiple queries and concurrent workers.

Parameters

- **query_id** – search query, find `k_nearest` neighbors of this DHTID
- **initial_nodes** – nodes used to pre-populate beam search heap, e.g. `[my_own_DHTID, ...maybe_some_peers]`
- **beam_size** – beam search will not give up until it exhausts this many nearest nodes (to `query_id`) from the heap Recommended value: A beam size of `k_nearest * (2-5)` will yield near-perfect results.
- **get_neighbors** – A function that returns neighbors of a given node and controls beam search stopping criteria. `async def get_neighbors(node: DHTID) -> neighbors_of_that_node: List[DHTID], should_continue: bool` If `should_continue` is `False`, beam search will halt and return `k_nearest` of whatever it found by then.
- **visited_nodes** – beam search will neither call `get_neighbors` on these nodes, nor return them as nearest

Returns

a list of `k` nearest nodes (nearest to farthest), and a set of all visited nodes (including `visited_nodes`)

 [latest](#) ▼

```
async hivemind.dht.traverse.traverse_dht(queries: Collection[DHTID], initial_nodes: List[DHTID],  
beam_size: int, num_workers: int, queries_per_call: int, get_neighbors: Callable[[DHTID, Collection[DHTID]],  
Awaitable[Dict[DHTID, Tuple[Tuple[DHTID], bool]]]], found_callback: Optional[Callable[[DHTID, List[DHTID],
```

`Set[DHTID]], Awaitable[Any]] = None, await_all_tasks: bool = True, visited_nodes: Optional[Dict[DHTID, Set[DHTID]]] = ()) → Tuple[Dict[DHTID, List[DHTID]], Dict[DHTID, Set[DHTID]]]` [\[source\]](#)

Search the DHT for nearest neighbors to :queries: (based on DHTID.xor_distance). Use get_neighbors to request peers. The algorithm can reuse intermediate results from each query to speed up search for other (similar) queries.

Parameters

- **queries** – a list of search queries, find beam_size neighbors for these DHTIDs
- **initial_nodes** – nodes used to pre-populate beam search heap, e.g. [my_own_DHTID, ...maybe_some_peers]
- **beam_size** – beam search will not give up until it visits this many nearest nodes (to query_id) from the heap
- **num_workers** – run up to this many concurrent get_neighbors requests, each querying one peer for neighbors. When selecting a peer to request neighbors from, workers try to balance concurrent exploration across queries. A worker will expand the nearest candidate to a query with least concurrent requests from other workers. If several queries have the same number of concurrent requests, prefer the one with nearest XOR distance.
- **queries_per_call** – workers can pack up to this many queries in one get_neighbors call. These queries contain the primary query (see num_workers above) and up to queries_per_call - 1 nearest unfinished queries.
- **get_neighbors** – A function that requests a given peer to find nearest neighbors for multiple queries `async def get_neighbors(peer, queries) -> {query1: ([nearest1, nearest2, ...], False), query2: ([...], True)}` For each query in queries, return nearest neighbors (known to a given peer) and a boolean “should_stop” flag If should_stop is True, traverse_dht will no longer search for this query or request it from other peers. The search terminates iff each query is either stopped via should_stop or finds beam_size nearest nodes.
- **found_callback** – if specified, call this callback for each finished query the moment it finishes or is stopped More specifically, run `asyncio.create_task(found_callback(query, nearest_to_query, visited_for_query))` Using this callback allows one to process results faster before traverse_dht is finishes for all queries. It is guaranteed that found_callback will be called exactly once on each query in queries.
- **await_all_tasks** – if True, wait for all tasks to finish before returning, otherwise returns after finding nearest neighbors and finishes the remaining tasks (callbacks and queries to known-but-unvisited nodes)
- **visited_nodes** – for each query, do not call get_neighbors on these nodes, nor return them among nearest.

Note

the source code of this function can get tricky to read. Take a look at *simple_traverse_dht* function for reference. That function implements a special case of *traverse_dht* with a single query and one worker.

Returns

a dict of nearest nodes, and another dict of visited nodes
nearest nodes: { query -> a list of up to beam_size nearest nodes, ordered nearest-first }
visited nodes: { query -> a set of all nodes that received requests for a given query }

Hivemind DHT

In order to coordinate, hivemind peers form a Distributed Hash Table: distributed “dictionary” where each peer can store and get values. To initialize the first DHT node, run

```
from hivemind import DHT, get_dht_time

dht = DHT(start=True)
# create the first DHT node that listens for incoming connections from localhost only

print("For incoming connections, use:", dht.get_visible_maddrs())
```

You can now start more peers that connect to an existing DHT node using its listen address:

```
dht2 = DHT(initial_peers=dht.get_visible_maddrs(), start=True)
```

Note that `initial_peers` contains the address of the first DHT node. This implies that the new node will share the key-value data with the first node, **as well as any other nodes connected to it**. When the two nodes are connected, subsequent peers can use any one of them (or both) as `initial_peers` to connect to the shared “dictionary”.

Store/get operations

Once the DHT is formed, all participants can `dht.store` key-value pairs in the DHT and `dht.get` them by key:

```
# first node: store a key-value pair for 600 seconds
store_ok = dht.store('my_key', ('i', 'love', 'bees'),
                    expiration_time=get_dht_time() + 600)

# second node: get the value stored by the first node
value, expiration = dht2.get('my_key', latest=True)
assert value == ('i', 'love', 'bees')
```

As you can see, each value in a hivemind DHT is associated with an expiration time, computed current `get_dht_time()` with some offset. This expiration time is used to cleanup old data and resolve write conflicts: DHT nodes always prefer values with higher expiration time and may delete any value past its expiration.

Values with subkeys

Hivemind DHT also supports a special value type that is itself a dictionary. When nodes store such a value, they add sub-keys to the dictionary instead of overwriting it.

Consider an example where three DHT nodes want to find out who is going to attend the party:

```
alice_dht = DHT(initial_peers=dht.get_visible_maddrs(), start=True)
bob_dht = DHT(initial_peers=dht2.get_visible_maddrs(), start=True)
carol_dht = DHT(initial_peers=alice_dht.get_visible_maddrs(), start=True)

# first, each peer stores a subkey for the same key
alice_dht.store('party', subkey='alice', value='yes', expiration_time=get_dht_time() + 600)
bob_dht.store('party', subkey='bob', value='yes', expiration_time=get_dht_time() + 600)
carol_dht.store('party', subkey='carol', value='no', expiration_time=get_dht_time() + 600)

# then, any peer can get the full list of attendees
attendees, expiration = alice_dht.get('party', latest=True)
print(attendees)
# {'alice': ValueWithExpiration(value='yes', expiration_time=1625504352.2668974),
#  'bob': ValueWithExpiration(value='yes', expiration_time=1625504352.2884178),
#  'carol': ValueWithExpiration(value='no', expiration_time=1625504352.3046832)}
```

When training over the Internet, some `dht.get/store` requests may run for hundreds of milliseconds and even seconds. To minimize the wait time, you can call these requests asynchronously via `dht.store/get/run_coroutine(..., return_future=True)` __. This will run the corresponding command in background and return a [Future-like](#) object that can be awaited. Please also note that the returned future is compatible with `asyncio` (i.e. can be awaited inside the event loop).

For more details on DHT store/get and expiration time, please refer to the [documentation for DHT and DHTNode](#)

Running across the Internet

By default, DHT nodes are only accessible from your localhost. In order to run with multiple geographically distributed computers, one must connect DHT to a global network. There are two ways to achieve this.

The recommended approach is to grow the network from one or several initial peers. These can be any computers with a public IP address that are always online. Each of these peers should simply create `hivemind.DHT` and set it to accept incoming connections from the internet:

```
import hivemind
dht = hivemind.DHT(
    host_maddrs=["/ip4/0.0.0.0/tcp/0", "/ip4/0.0.0.0/udp/0/quic"],
    start=True)

print('\n'.join(str(addr) for addr in dht.get_visible_maddrs()))
print("Global IP:", hivemind.utils.networking.choose_ip_address(dht.get_visible_maddrs()))
```

Running this code will print several, typically, 4 or 6 strings of the following form (example):

```
/ip4/185.185.123.124/tcp/40615/p2p/QmaVTB2LwayToK2rzMkaCbKCaH7nF2rTHIS0IS0AN0EXAMPLE
/ip4/127.0.0.1/tcp/40615/p2p/QmaVTB2LwayToK2rzMkaCbKCaH7nF2rTHIS0IS0AN0EXAMPLE
/ip4/185.185.123.124/udp/40346/quic/p2p/QmaVTB2LwayToK2rzMkaCbKCaH7nF2rTHIS0IS0AN0EXAMPLE
/ip4/127.0.0.1/udp/40346/quic/p2p/QmaVTB2LwayToK2rzMkaCbKCaH7nF2rTHIS0IS0AN0EXAMPLE
Global IP: 185.185.123.124
```

The lines that contain addresses that other nodes can use to connect to the network:

- `127.0.0.1` or `192.168.X.Y` are only accessible from your computer or local network, respectively.
- The remaining address is **global** (`185.185.123.124` in the example, yours will be different).

To connect a new peer to the network, you should specify `initial_peers` as the addresses that correspond to the public IP:

```
import hivemind
dht = hivemind.DHT(
    host_maddrs=["/ip4/0.0.0.0/tcp/0", "/ip4/0.0.0.0/udp/0/quic"],
    initial_peers=[
        "/ip4/185.185.123.124/tcp/40615/p2p/QmaVTB2LwayToK2rzMkaCbKCaH7nF2rTHIS0IS0AN0EXAMPLE",
        "/ip4/185.185.123.124/udp/40346/quic/p2p/QmaVTB2LwayToK2rzMkaCbKCaH7nF2rTHIS0IS0AN0EXAMPLE",
    ], start=True)
```

That's it, now the two DHT nodes are connected. If you connect additional peers to the network, you only need to specify one (or a subset) of peers as `initial_peers`. I

operates behind a restrictive firewall, you may find it beneficial to set `client_moue=true`.  latest 

case, the DHT instance will access others, but it will not announce that other peers can connect to it.

Another (experimental) way is to use [IPFS](#): a global decentralized network for file storage. We are not storing any files here: instead, we can use IPFS nodes to help hivemind peers find each other. To use this strategy, set `use_ipfs=True` in each DHT node you create. This allows you to connect DHT multiple even if all of them are behind NAT. However, this strategy may be unreliable and depend heavily on the availability of public IPFS nodes.

To learn more about the network address format, read [libp2p addressing](#) For an example of how to set up DHT in a distributed training experiment, see [examples/albert](#)